Malware Analysis: Setup.exe

This is a 64-bit Windows program written in Go. It works as a sneaky dropper and loader. Based on the file, registry, and process activity seen in the sandbox, its job is to set up its own encryption, drop a disguised payload, and spawn child processes to carry out the real attack.

Go binaries are popular with attackers right now because they compile into huge files with unusual memory structures, which trips up a lot of traditional antivirus engines.

1. Structure

File type: 64-bit Windows executable (PE32+ / AMD64).

Built with: the Go build ID string baked into the binary, and from the disassembly using 64-bit registers like RBP, RSP, and RAX along with standard Go stack frames.

File names: It shows up under both generic names like `Setup.exe` and randomized ones like `kvrlriebsz.exe`, `dxcucjfvpb.exe`, `tiuaupwrdy.exe`, and `ldshq.exe`. The random names are probably there to dodge hash based detection, since a different filename per victim breaks simple static signatures.

Sections in the file:

- `.text`: the Go runtime plus the compiled code
- `.rdata` and `.data`: constants, string tables, global variables
- `.pdata` and `.xdata`: exception handling tables for 64-bit stack unwinding
- `.symtab`: a symbol table, which is a classic Go fingerprint. It keeps internal function names around, so reversing it with Go-aware tools is easier than usual

2. FUNCTIONALITY

A. Makes sure only one copy runs

It creates a mutex named `Glasikprostik` under `\Sessions\1\BaseNamedObjects\`. When it starts, it checks for this mutex first. If it already exists, that means another copy is already running, so it just exits. This stops multiple infections from fighting over resources and drawing attention.

B. Checks its surroundings

It opens `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Internet Settings`, likely to read the proxy configuration so it knows how to route traffic if it needs to reach a command and control server.

It also opens `HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AppCompatFlags`, probably to check the exact OS build and see what compatibility shims are available, since those can sometimes be abused for UAC bypass or persistence.

The binary also contains fragments of strings from Go's internal CPU detection package. This points to the malware checking whether the processor supports hardware accelerated cryptography like AES-NI, so it can decrypt its payload faster.

C. Sets up encryption

The sandbox shows it talking to `\Device\KsecDD`, the kernel's secure device for generating random numbers and keys, and loading `Sspicli.dll` and `bcryptprimitives.dll`, Microsoft's modern crypto library.

Put together, this looks like the malware asking Windows to securely generate encryption keys, which it then uses to decrypt a hidden payload sitting inside its own `.text` or `.rdata` section. That's how it pulls the real malware out of its Go shell, in memory.

D. Drops the payload disguised as something else

The sandbox logs show a file landing at a path that looks like `/msdownload/update/v3/static/trustedr.../authrootstl.cab`, inside a fake folder structure meant to look like a legitimate Windows Update temp directory.

`authrootstl.cab` is normally a real Microsoft file that lists trusted root certificates. Naming the payload this way is meant to trick a quick look, whether human or automated, into thinking it's a harmless update file.

E. Spawns a child process to run it

The sandbox shows two `Setup.exe` processes in the tree. The first one, run from the desktop, creates a second process, most likely through `CreateProcessW` called via Go's `os/exec` package.

This second process is probably how the malware survives if the original `Setup.exe` gets killed by a security tool. The child process is likely the one that actually does the damage, whether that's stealing data, deploying ransomware, or something else.

Execution Order

1. The user runs `Setup.exe` from the desktop.
2. It creates the `Glasikprostik` mutex to make sure only one copy runs.
3. It checks Internet Settings and AppCompatFlags in the registry for proxy info and OS version.
4. It checks CPU features and talks to `\Device\KsecDD` and `bcryptprimitives.dll` to generate encryption keys.
5. It decrypts its internal Go payload using those keys.
6. It writes the decrypted payload to disk, disguised as `authrootstl.cab` in a fake Windows Update folder.
7. It spawns a second process to run the payload, giving it persistence and separating the malicious process from the original dropper.